PAN-ATLANTIC UNIVERSITY

# Cargo and Data Types

CSC 101 Practical · Week 3

2026/2027 Session

Pan-Atlantic University

# Today's Practical

**Part 1**

## Meet Cargo

Rust's build system and package manager. Create, build and run a project with it.

**Part 2**

## Data types

Integers, floats, booleans and characters. Practices 1 to 7.

**Part 3**

## Mutability

Variables are read-only until you say otherwise. Ends with a debugging exercise.

PART 1 OF 3

# Meet Cargo

Rust's build system and package manager. You create, build and run a real project.



# What is Cargo?

### Rust's build system and package manager

Most Rustaceans manage their projects with Cargo because it handles so much for you: building your code, downloading the libraries your code depends on, and building those libraries too.

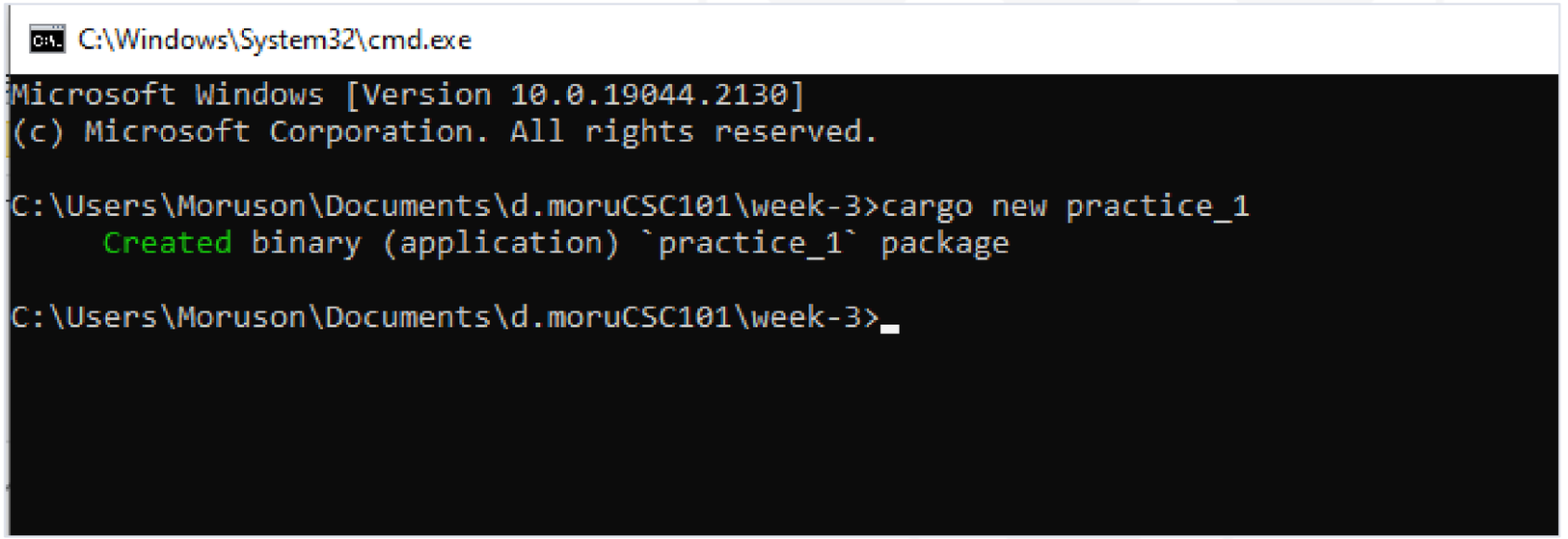
### You already have it

Cargo comes installed with Rust if you used the official installers last week.

### Check for yourself

```
$ cargo --version
```

# Create a Project with Cargo



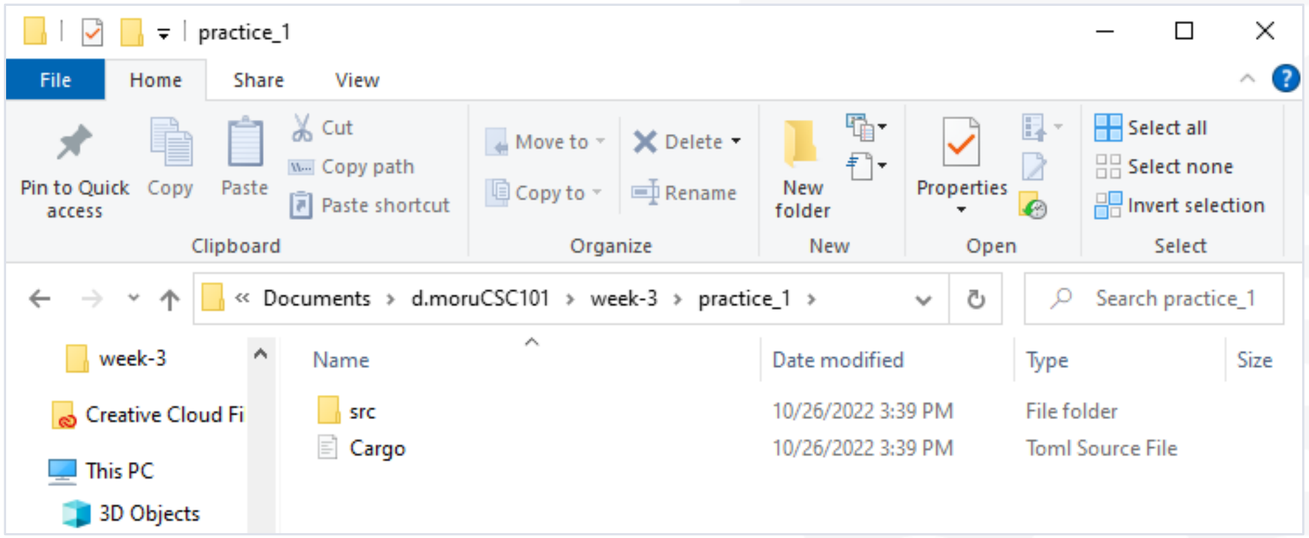

1. **Go to this week's folder in your repo**
   `cd Documents\<repo>\week-3`

2. **Ask Cargo for a new project**
   `cargo new practice_1`

3. **Cargo confirms**
   Created binary (application) practice_1 package.

# What Cargo Just Made

**THE NEW FOLDER**




**CARGO.TOML**



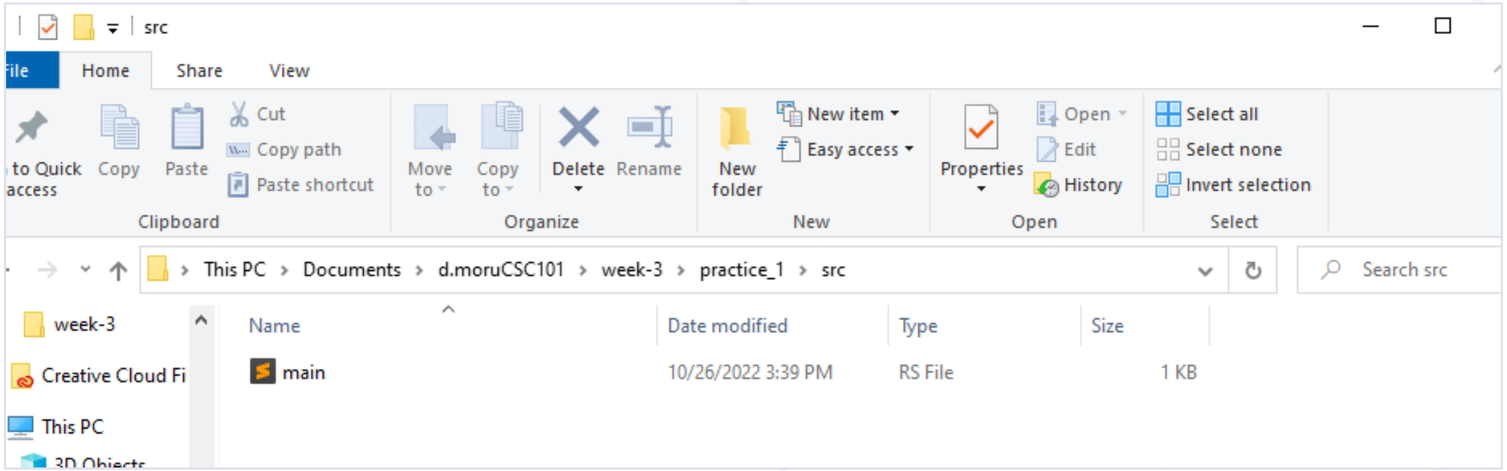

1. **A practice_1 folder appears**
   Cargo created it for you.

2. **Open Cargo.toml with Notepad**
   .toml is the project's configuration file: name, version, edition.

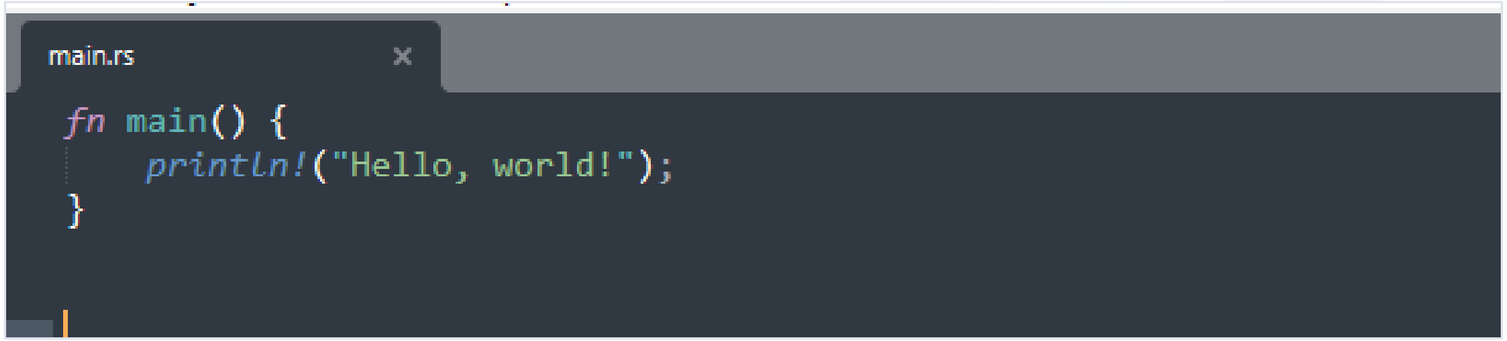
3. **Leave it as it is**
   You will edit this file in later weeks, not today.

# Inside the src Folder

THE SRC FOLDER




MAIN.RS

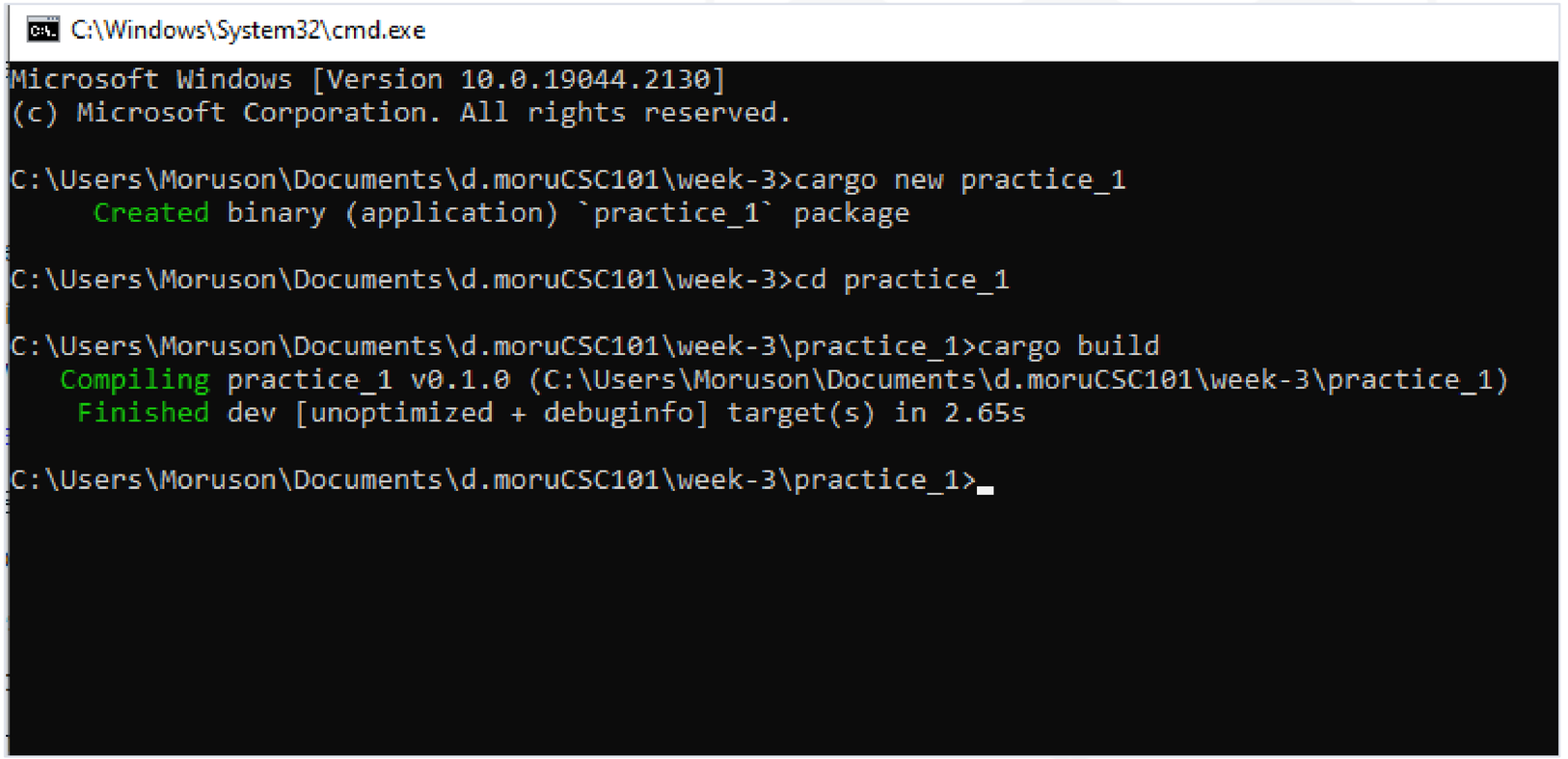



1. **Open the src folder**

2. **Open main.rs**
   Your code lives here from now on.

3. **Cargo already wrote a program**
   `println!("Hello, world!");`

# Build the Project

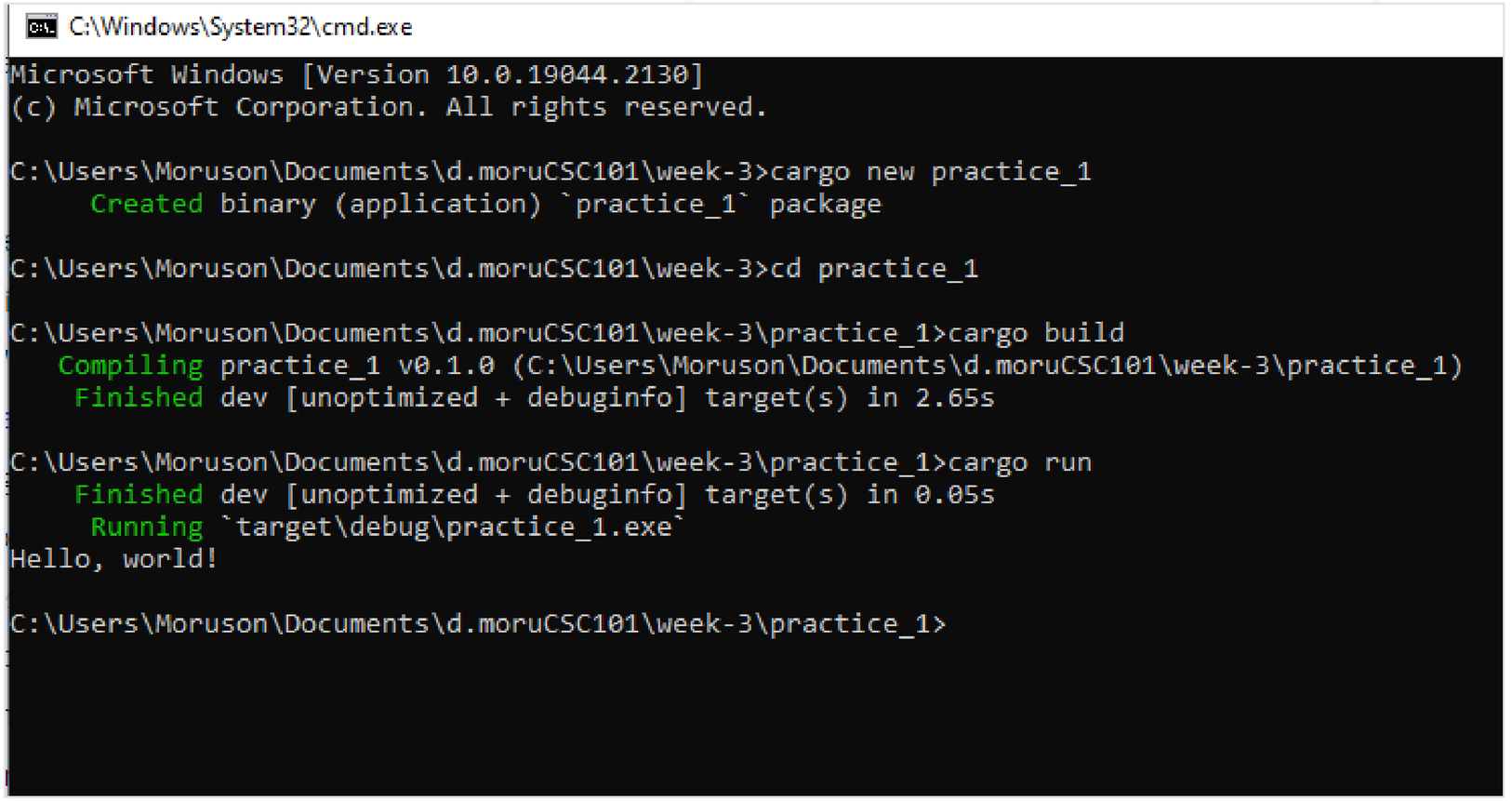



1. **Create, then step inside**
   `cargo new practice_1`

2. `cd practice_1`

3. **Build it**
   `cargo build`
   Finished dev target(s): it compiled.

# Run the Project

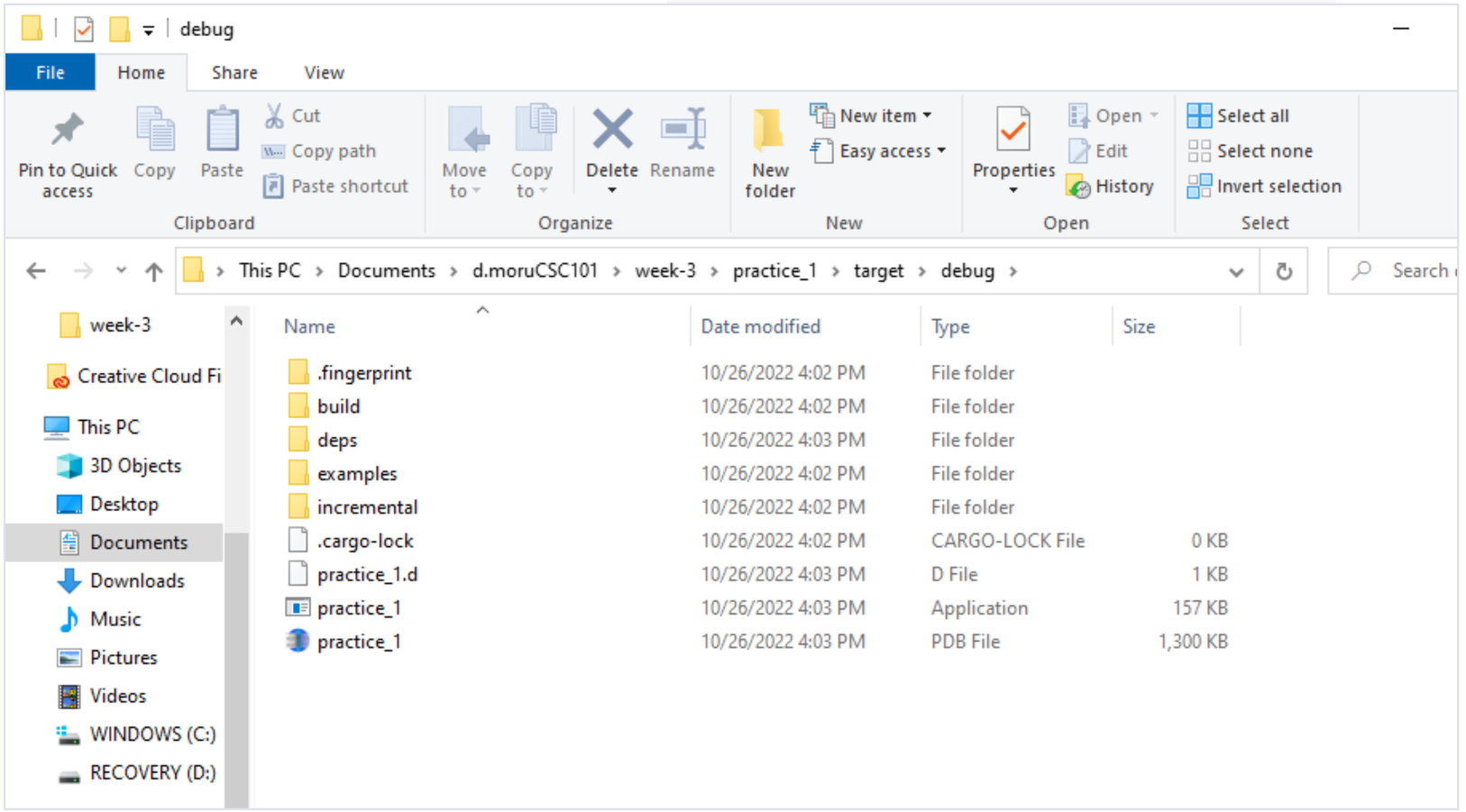



1. **One command does it all**
   `cargo run`

2. **What it actually runs**
   `target\debug\practice_1.exe`
   cargo run builds first if anything changed, then runs the result.

**Tip:** Compare with Week 2: rustc + the .exe by hand. Cargo folds both into one command.

# Where the Compiled File Lives




1. **Open the project in Explorer**
   `practice_1\target\debug`

2. **There is your program**
   `practice_1.exe`
   The same kind of .exe rustc gave you, just filed away by Cargo.

PART 2 OF 3

# Data Types

Integers, floats, booleans and characters, and what the compiler does with them.



# What is a Data Type?

### Rust is statically typed

Every value in Rust has a data type. The compiler can infer it from the value you assign, which is why Week 2's code worked without naming any types.

### The type system

It represents all the kinds of values the language supports, and it checks every value is valid before it is stored or touched by the program.

### Why it matters

Code behaves as expected, and you get richer hints and documentation from your tools for free.

# The Four Scalar Types

A scalar type holds a single value, like 10, 3.14 or 'c'. Rust has four primary ones.

**Integer**
whole numbers

**Floating-point**
decimals

**Boolean**
true or false

**Character**
one symbol

*Practices 1 to 7 walk through each of these in code.*

# Practice 1: One of Each

Everything from Week 2, now living inside a Cargo project.

**THE CODE**

main.rs

```rust
fn main() {
    let school_string = "School of Science and Technology";  // string type
    let rating_float = 5.0;                 // float type
    let is_growing_boolean = true;       // boolean type
    let department_integer = 3;           // integer type


    println!("School Name is: {}",school_string);
    println!("School rating on 5 is: {}",rating_float);
    println!("School is growing : {}",is_growing_boolean);
    println!("Number of Departments : {}",department_integer);
}
```

**THE OUTPUT**

```
C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_1>cargo build
   Compiling practice_1 v0.1.0 (C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_1)
    Finished dev [unoptimized + debuginfo] target(s) in 2.41s

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_1>cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.04s
     Running `target\debug\practice_1.exe`
School Name is: School of Science and Technology
School rating on 5 is: 5
School is growing : true
Number of Departments : 3

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_1>
```

1. **New project, new habit**
   `week-3\practice_1\src\main.rs`

2. **Four variables, four types**
   A string, a float, a boolean, an integer. The comments name each one.

3. **Build and run with Cargo**
   `cargo build`, `cargo run`

# Integer Data Type

An integer is a number without a fractional component: a whole number. Signed integers (i) can hold negative and positive values. Unsigned integers (u) hold positive values only.

| Size | Signed | Unsigned |
|---|---|---|
| 8 bit | `i8` | `u8` |
| 16 bit | `i16` | `u16` |
| 32 bit | `i32` | `u32` |
| 64 bit | `i64` | `u64` |
| 128 bit | `i128` | `u128` |

# Practice 2: Integers

Integers default to i32 unless you say otherwise.

**THE CODE**

main.rs

```
fn main() {
    let result = 10;    // i32 by default
    let age:u32 = 20;
    let sum:i32 = 5 - 15;

    println!("Result value is {} ",result);
    println!("Age is {} ",age);
    println!("Sum is {} ",sum);

}
```

**THE OUTPUT**

```
C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_2>cargo build
   Compiling practice_2 v0.1.0 (C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_2)
    Finished dev [unoptimized + debuginfo] target(s) in 2.75s

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_2>cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.05s
     Running `target\debug\practice_2.exe`
Result value is 10
Age is 20
Sum is -10

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_2>
```

1. **No type stated? It is i32**
   `let result = 10;`

2. **Unsigned, stated yourself**
   `let age: u32 = 20;`

3. **Signed can go negative**
   `let sum: i32 = 5 - 15;`
   The output prints -10, only possible because i32 is signed.

# Practice 3: Floats

Float types come in two sizes: f32 is single precision, f64 is double. The default is f64.

main.rs

```rust
fn main() {
    let result = 10.00;         //f64 by default
    let interest:f32 = 8.35;
    let cost:f64 = 15000.600;  //double precision

    println!("result value is {}",result);
    println!("interest is {}",interest);
    println!("cost is {}",cost);
}
```

1. **No type stated? It is f64**
   `let result = 10.00;`

2. **Single precision if you ask**
   `let interest: f32 = 8.35;`

3. **Double precision**
   `let cost: f64 = 15000.600;`

# Practice 4: Booleans

Boolean types hold one of two values: true or false.

**THE CODE**

main.rs

```
fn main() {
    let isgood:bool = true;
    println!("PAU is a good university? {}",isgood);

}
```

**THE OUTPUT**

```
C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_4>cargo build
   Compiling practice_4 v0.1.0 (C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_4)
    Finished dev [unoptimized + debuginfo] target(s) in 2.18s

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_4>cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.05s
     Running `target\debug\practice_4.exe`
PAU is a good university? true

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_4>
```

1. **Declare with the bool keyword**
   `let isgood: bool = true;`

2. **Two possible values**
   true or false, nothing else.

3. **The output answers the question**
   PAU is a good university? true.

# Practice 5: Characters and Strings

The character data type supports numbers, alphabets and special characters. Declare it with the char keyword.

main.rs

```rust
fn main() {
    let special_character = '@'; //default
    let alphabet:char = 'B';
    let surname = "Bolaji";
    let first_name = "Michael";
    let middle_name = "Gboyega";

    println!(" ");
    println!("Special character: {}",special_character);
    println!("Alphabet: {}",alphabet);
    println!("Name: {} {} {}",surname, middle_name, first_name);
}
```

1. **char uses single quotes**
   `let alphabet: char = 'B';`

2. **Strings use double quotes**
   `let surname = "Bolaji";`

3. **char is bigger than ASCII**
   Numbers, alphabets, special characters like '@', and well beyond.

# Practice 6: Type Casting

Automatic type casting is not allowed in Rust. Watch what happens when an integer is assigned to a float variable.

**THE CODE**

main.rs

```rust
fn main() {
    let interest:f32 = 8;   // integer assigned to float variable
    println!("interest is {}",interest);
```

**THE OUTPUT**

```
error[E0308]: mismatched types
 --> src\main.rs:2:23
  |
2 |     let interest:f32 = 8;   // integer assigned to float variable
  |                  ---   ^
  |                  |     |
  |                  |     expected `f32`, found integer
  |                  |     help: use a float literal: `8.0`
  |                  expected due to this

For more information about this error, try `rustc --explain E0308`.
error: could not compile `practice_4` due to 2 previous errors

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_4>
```

1. **An integer into a float variable**
   `let interest: f32 = 8;`

2. **The compiler refuses**
   error[E0308]: mismatched types. Expected f32, found integer.

3. **And hands you the fix**
   `use a float literal: 8.0`

# Practice 7: Number Separator

For readability, big numbers can take a visual separator: 50,000 is written 50_000.

**THE CODE**

main.rs

```rust
fn main() {
    let float_with_separator = 11_000.555_001;
    println!("float value {}",float_with_separator);

    let int_with_separator = 50_000;
    println!("int value {}",int_with_separator);
}
```

**THE OUTPUT**

```
C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_7>cargo build
   Finished dev [unoptimized + debuginfo] target(s) in 0.05s

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_7>cargo run
   Finished dev [unoptimized + debuginfo] target(s) in 0.04s
    Running `target\debug\practice_7.exe`
float value 11000.555001
int value 50000

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_7>
```

1. **Underscores instead of commas**
   `let int_with_separator = 50_000;`

2. **Works inside floats too**
   `11_000.555_001`

3. **Purely visual**
   The value is unchanged. 50_000 is exactly 50000.

PART 3 OF 3

# Mutability

Why Rust variables are read-only by default, and the keyword that changes it.



# Practice 8: Immutable by Default

By default, Rust variables are immutable: read-only. Once a value is bound to a name, it cannot be changed.

**THE CODE**

main.rs

```rust
fn main() {
    let fees = 25_000;
    println!("fees is {} ",fees);

    fees = 35_000;
    println!("fees changed is {}",fees);
}
```

**THE OUTPUT**

```
C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_8>cargo build
   Compiling practice_8 v0.1.0 (C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_8)
error[E0384]: cannot assign twice to immutable variable `fees`
 --> src\main.rs:5:4
  |
2 |    let fees = 25_000;
  |        ----
  |        |
  |        first assignment to `fees`
  |        help: consider making this binding mutable: `mut fees`
...
5 |    fees = 35_000;
  |    ^^^^^^^^^^^^ cannot assign twice to immutable variable

For more information about this error, try `rustc --explain E0384`.
error: could not compile `practice_8` due to previous error
```

1. **fees is bound once**
   `let fees = 25_000;`

2. **A second assignment fails**
   error[E0384]: cannot assign twice to immutable variable.

3. **The compiler already knows the cure**
   help: consider making this binding mutable: mut fees.

# Practice 9: The mut Keyword

To make a variable mutable, prefix its name with the mut keyword.

**THE CODE**

main.rs

```rust
fn main() {
    let mut fees:i32 = 25_000;
    println!("fees is {} ",fees);

    fees = 35_000;
    println!("fees changed is {}",fees);
}
```

**THE OUTPUT**

```
C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_9>cargo build
   Compiling practice_9 v0.1.0 (C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_9)
    Finished dev [unoptimized + debuginfo] target(s) in 3.13s

C:\Users\Moruson\Documents\d.moruCSC101\week-3\practice_9>cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.05s
     Running `target\debug\practice_9.exe`
fees is 25000
fees changed is 35000
```

1. **One word changes everything**
   `let mut fees: i32 = 25_000;`

2. **Reassignment now works**
   `fees = 35_000;`

3. **The output shows both values**
   fees is 25000, then fees changed is 35000.

# Practice 10: Debug This Program

Your closing exercise. This program has bugs planted in it: your job is to make it compile and run.

main.rs

```rust
fn main() {
    // addition
    let sum = 5550 + 7310
    println!("The sum of 5550 and 7310 = {}", sum);

    // subtraction
    let difference:u32 = 95.5 - 4.3;
    println!("The difference of 95.5 and 4.3 = ()", difference);

    // multiplication
    let product:f32 = 4 * 30;
    println!("The multiple of 4 and 30 = {}", product);

    // division
    let quotient = 56.7 / 32.2;
    println!("The division of 56.7 and 32.2 = {}, quotient");

    // remainder
    let remainder = 43 % 5
    println!("The remainder of 43 and 5 = {}", remainder);
}
```

1. **Five operations**
   Addition, subtraction, multiplication, division, remainder.

2. **Some type declarations are wrong**
   On purpose. It will not compile as written.

3. **Fix it until it runs**
   Read the compiler errors. They name the line and usually the cure.

PAN-ATLANTIC UNIVERSITY

# Questions?

Homework: Practices 1 to 10 live in week-3.
Push them before next class.